

Data Acquisition, Cleaning, and Preprocessing

A Comprehensive Report on Preparing the Titanic Passenger Dataset for Analysis

Prepared by: Dhanya B K

B.E. — Artificial Intelligence and Machine Learning

Vivekananda College of Engineering and Technology, Puttur, Karnataka

Week 1 Task — Data Acquisition, Cleaning, and Preprocessing

Contents

- 1. Objective
- 2. Dataset Selection & Data Collection Method
- 3. Initial Data Exploration
- 4. Identifying Missing Values, Inconsistencies & Outliers
- 5. Data Cleaning Procedure
- 6. Preprocessing for Downstream Analysis
- 7. Challenges Faced and How They Were Overcome
- 8. Impact of Preprocessing on Subsequent Analysis
- 9. Conclusion

1. Objective

This report documents the end-to-end process of acquiring a publicly available dataset, exploring its structure, identifying data quality issues, and applying cleaning and preprocessing techniques using Python. The goal is to produce an analysis-ready dataset while transparently recording every decision made along the way — including the reasoning behind each choice and the trade-offs it involves.

The dataset chosen for this exercise is the Titanic passenger dataset, a widely used public dataset that realistically exhibits the three core categories of data quality problems this task focuses on: missing values, outliers, and inconsistent/erroneous entries.

2. Dataset Selection & Data Collection Method

Dataset: Titanic Passenger Dataset (891 passenger records, 12 original columns).

Source: Public GitHub mirror of the dataset —

<https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv> (a widely referenced open mirror of the original Kaggle Titanic competition dataset).

Collection method: The CSV file was programmatically downloaded via an HTTP request in the working environment and loaded into a pandas DataFrame for analysis. No manual data entry or scraping was required, since the dataset is distributed as a ready-to-use, structured CSV file.

Why this dataset?

- It is small enough to explore thoroughly within the scope of this task, yet large enough (891 rows) to be statistically meaningful.
- It contains a realistic mix of numeric (Age, Fare), categorical (Sex, Embarked, Pclass) and free-text (Name, Ticket, Cabin) columns.
- It has genuine, well-documented missing values (Age, Cabin, Embarked) and skewed/outlier-prone columns (Fare), making it ideal for demonstrating cleaning techniques rather than needing to synthesize artificial problems.

3. Initial Data Exploration

Before any cleaning, the raw dataset was explored to understand its structure, data types, and overall quality.

```
import pandas as pd

df = pd.read_csv('titanic.csv')
print(df.shape)
print(df.dtypes)
print(df.describe())
print(df.isnull().sum())
```

Key findings

Property	Value
Rows	891
Columns	12
Duplicate rows	0

Numeric columns	PassengerId, Survived, Pclass, Age, SibSp, Parch, Fare
Categorical / text columns	Name, Sex, Ticket, Cabin, Embarked

The overall survival rate in the raw data was approximately 38.4% (342 of 891 passengers), with a class distribution skewed toward 3rd class (491 passengers) compared to 1st (216) and 2nd (184).

4. Identifying Missing Values, Inconsistencies & Outliers

4.1 Missing values

Column	Missing Count	Missing %
Cabin	687	77.10%
Age	177	19.87%
Embarked	2	0.22%

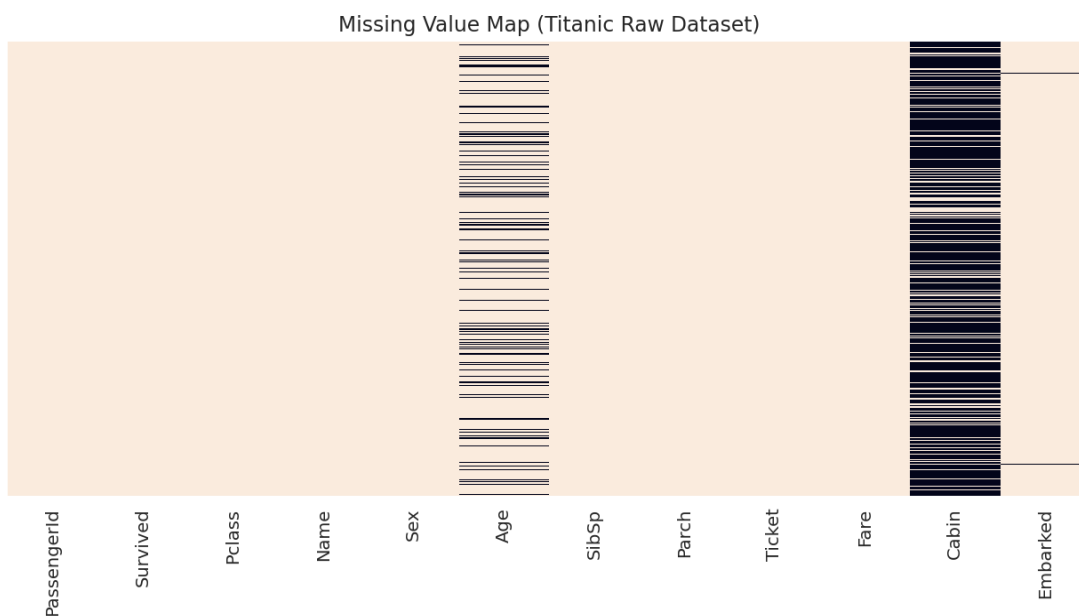


Figure 1 — Missing value map across the raw dataset (Cabin is sparsely populated; Age has moderate gaps).

4.2 Outlier detection (IQR method)

Outliers were quantified using the Interquartile Range (IQR) method: values falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ were flagged as statistical outliers.

```
def iqr_outliers(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - 1.5*iqr, q3 + 1.5*iqr
    mask = (series < lower) | (series > upper)
    return mask.sum(), lower, upper
```

Column	IQR Bounds	Outliers Found	% of Data
--------	------------	----------------	-----------

Age	[-6.69, 64.81]	11	1.23%
Fare	[-26.72, 65.63]	116	13.02%
SibSp	[-1.50, 2.50]	46	5.16%
Parch	[0.00, 0.00]	213	23.91%

Fare stood out as the column with the most impactful outliers — 13% of records, including extreme values up to 512.33, far above the typical fare range. SibSp and Parch show high 'outlier' percentages largely because most passengers travelled with zero or one family member, making any larger family size register as statistically extreme even though it is a valid, meaningful value — this distinction mattered for deciding how to treat each column (see Section 5).

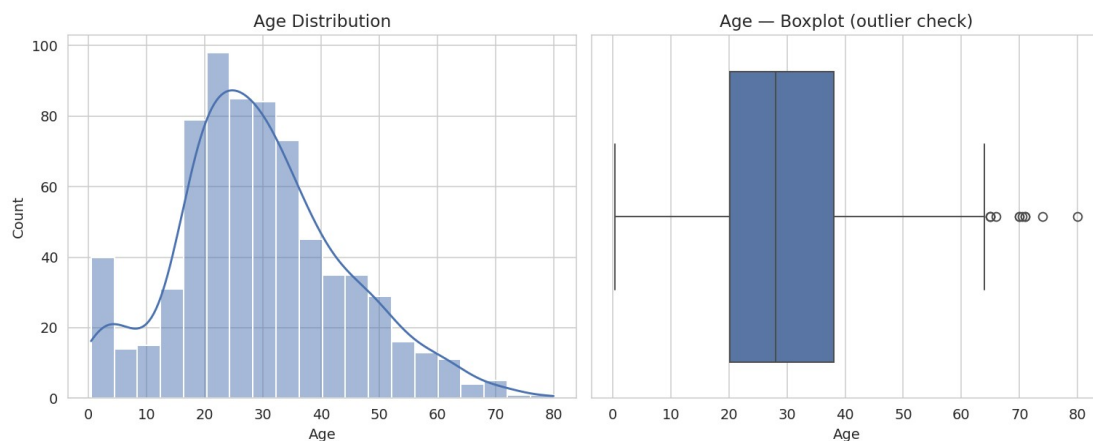


Figure 2 — Age distribution and boxplot: mild right skew, few extreme values.

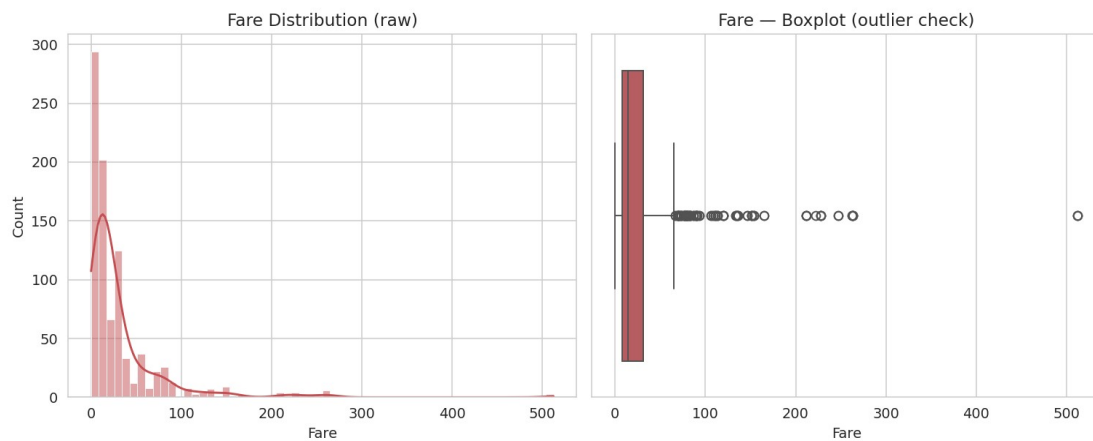


Figure 3 — Fare distribution and boxplot: strong right skew with high-value outliers.

4.3 Erroneous / inconsistent entries

- 15 records had Fare = 0 despite belonging to Pclass 1–3 — inconsistent with a paid passage and most likely a data-entry artifact (some of these correspond to crew or employee tickets historically misfiled in the passenger manifest).
- No duplicate rows were found in the raw data (checked via `df.duplicated()`).

- No negative or unrealistic Age values (e.g. ≤ 0 or > 100) were present, but Age was stored as a float even for whole-number ages, which is expected since a small number of infant ages are fractional (e.g. 0.42).

5. Data Cleaning Procedure

Each data quality issue was addressed with a technique chosen to preserve as much information as possible rather than defaulting to row deletion.

5.1 Embarked (2 missing values)

With only 2 missing values (0.22%), mode imputation was used — the missing entries were filled with the most frequent port, 'S' (Southampton).

```
mode_embarked = df['Embarked'].mode()[0]
df['Embarked'] = df['Embarked'].fillna(mode_embarked)
```

5.2 Age (177 missing values, ~20%)

A global median would ignore the fact that Age correlates with passenger title (e.g. 'Master' indicates a young boy, 'Mrs' an adult woman) and travel class. A title was first extracted from the Name field via regex, grouped into five standard categories (Mr, Mrs, Miss, Master, Rare), and Age was imputed using the median Age within each Pclass × Title group — a more representative estimate than a single dataset-wide median.

```
df['Title'] = df['Name'].str.extract(r',\s*([^\s]*)\s\.')
title_map = {'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs', 'Dr': 'Rare', ...}
df['Title'] = df['Title'].replace(title_map)

df['Age'] = df.groupby(['Pclass', 'Title'])['Age']\
    .transform(lambda s: s.fillna(s.median()))
```

5.3 Cabin (687 missing values, ~77%)

With over three-quarters of values missing, imputing Cabin directly would fabricate information with no reliable basis. Instead of dropping the column outright (which would discard a potentially useful signal — cabin availability itself correlates with ticket class and survival), it was converted into two derived, fully-populated features: a binary 'HasCabin' flag, and a 'Deck' category (the first letter of the cabin code, or 'Unknown' when missing).

```
df['HasCabin'] = df['Cabin'].notnull().astype(int)
df['Deck'] = df['Cabin'].str[0].fillna('Unknown')
df = df.drop(columns=['Cabin'])
```

5.4 Zero-value Fare entries

The 15 records with Fare = 0 were treated as erroneous rather than genuine free passage, and were replaced with the median fare of their respective passenger class.

```
median_fare_by_class = df[df['Fare']>0].groupby('Pclass')['Fare'].median()
for pclass, med in median_fare_by_class.items():
    mask = (df['Fare']==0) & (df['Pclass']==pclass)
    df.loc[mask, 'Fare'] = med
```

5.5 Fare outliers

Rather than deleting the 116 rows flagged as Fare outliers (13% of the dataset — too large a proportion to discard without introducing bias), values above the IQR upper bound were capped (winsorized) at that bound. This retains every passenger record while reducing the distortion extreme fares would otherwise cause in downstream statistics such as means and correlations.

```
q1, q3 = df['Fare'].quantile(0.25), df['Fare'].quantile(0.75)
upper_bound = q3 + 1.5*(q3-q1)
df['Fare'] = np.where(df['Fare'] > upper_bound, upper_bound, df['Fare'])
```

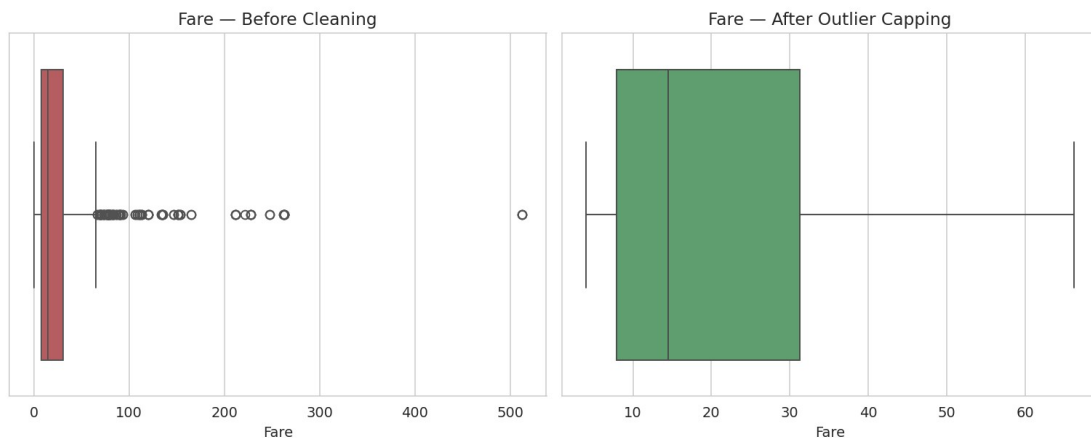


Figure 4 — Fare distribution before and after outlier capping.

5.6 SibSp and Parch — deliberately left untouched

Although the IQR method flagged a large share of SibSp and Parch values as 'outliers,' these represent genuine, meaningful family-size counts rather than data errors, so no capping or removal was applied. This illustrates an important principle followed throughout this task: a statistical outlier is not automatically an erroneous value, and each flagged case was evaluated on domain grounds before deciding how — or whether — to treat it.

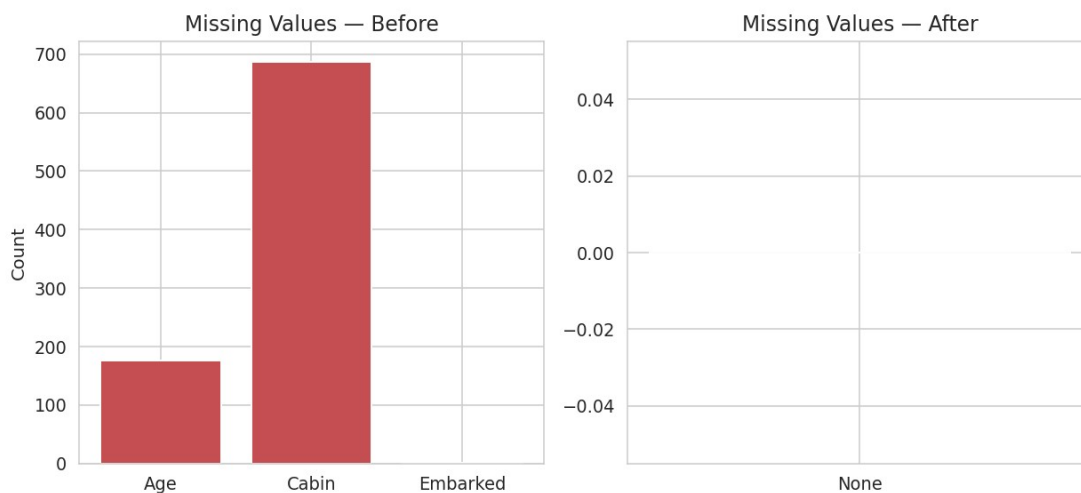


Figure 5 — Missing values before cleaning vs. after cleaning (zero remaining).

5.7 Result

After cleaning: 891 rows retained (no rows dropped), 0 missing values remaining, 0 duplicate rows, and Fare outliers controlled without loss of data. The cleaned dataset was saved as `titanic_cleaned.csv`.

6. Preprocessing for Downstream Analysis

With the data cleaned, a further preprocessing pass prepared it for statistical analysis or machine learning.

6.1 Feature engineering

- `FamilySize = SibSp + Parch + 1` (captures total family unit size in one variable).
- `IsAlone = 1` if `FamilySize == 1`, else 0 (a simplified, often more predictive signal than raw family size).
- Dropped `PassengerId`, `Name`, and `Ticket` — identifiers/free text with no direct analytical value once `Title` had already been extracted from `Name`.

```
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)
df = df.drop(columns=['PassengerId', 'Name', 'Ticket'])
```

6.2 Encoding categorical variables

`Sex` was binary-mapped (`male=0`, `female=1`). `Embarked`, `Title`, and `Deck` were one-hot encoded using `pandas.get_dummies` with `drop_first=True` to avoid the dummy-variable trap (perfect multicollinearity).

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(df, columns=['Embarked', 'Title', 'Deck'], drop_first=True)
```

6.3 Feature scaling

`Age`, `Fare`, and `FamilySize` were standardized (zero mean, unit variance) using scikit-learn's `StandardScaler`, since these features have very different natural ranges (`Age` in years vs. `Fare` in currency units) and many downstream algorithms (e.g. distance-based or gradient-based models) are sensitive to feature scale.

```
from sklearn.preprocessing import StandardScaler

numeric_cols = ['Age', 'Fare', 'FamilySize']
scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

6.4 Final preprocessed dataset

The final preprocessed dataset contains 891 rows and 24 columns, fully numeric and free of missing values, saved as `titanic_preprocessed.csv`.

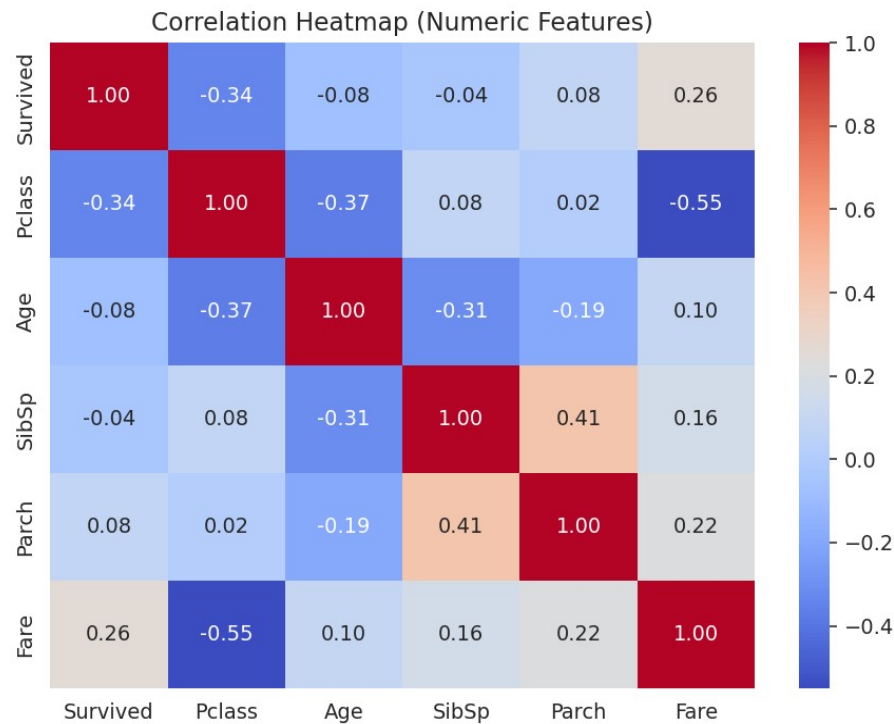


Figure 6 — Correlation heatmap of core numeric features after cleaning.

7. Challenges Faced and How They Were Overcome

Challenge 1 — Choosing an imputation strategy for Age

A simple global median would have been the easiest option, but it would erase meaningful variation (e.g. children vs. adults). This was resolved by engineering a Title feature from the Name column and imputing Age using $\text{Pclass} \times \text{Title}$ group medians, which better reflects the true underlying distribution.

Challenge 2 — Deciding what 'missing' means for Cabin

With 77% missing, standard imputation was not viable, and dropping the column entirely would discard the information that having a recorded cabin at all is meaningful. The resolution was to re-frame the problem: instead of asking 'what cabin did this passenger have,' the more answerable question 'do we know this passenger's cabin' was captured as a binary feature, alongside a coarser Deck category.

Challenge 3 — Distinguishing genuine outliers from meaningful extreme values

The IQR method flags any statistically extreme value, but not every flagged value is an error — SibSp and Parch outliers were legitimate large-family records, while Fare outliers reflected genuine data skew and a few misfiled zero-fare entries required separate treatment. This was resolved by inspecting each flagged column individually rather than applying a single blanket outlier-removal rule across the dataset.

Challenge 4 — Avoiding information loss while cleaning

Deleting every row with any issue would have discarded a large fraction of the dataset. Imputation, feature derivation (HasCabin, Deck), and capping (rather than removing) outliers were used throughout so that all 891 original passenger records were retained in the final dataset.

8. Impact of Preprocessing on Subsequent Analysis

- Imputing Age using group medians avoids artificially shrinking variance the way a single global constant would, preserving more realistic downstream statistics (e.g. correlation with Survived) — though any imputation still slightly understates true variability compared to fully observed data.
- Capping Fare outliers reduces the influence of a handful of extreme values on models sensitive to scale (e.g. linear regression, k-NN, clustering) and prevents the mean/standard deviation from being pulled disproportionately by a small number of high fares.
- Converting Cabin into HasCabin/Deck retains a historically meaningful signal (cabin data availability correlates with passenger class and survival) that would otherwise be lost entirely by dropping the column.
- One-hot encoding expands dimensionality (12 → 24 columns), which downstream analyses should account for — e.g. by using regularization in linear models to avoid overfitting on the added categorical indicator columns.
- Standardizing numeric features makes coefficients and distance metrics directly comparable across Age, Fare, and FamilySize, which is a prerequisite for many common Python ML workflows but is not needed for tree-based models (e.g. decision trees, random forests), which are scale-invariant.

Overall, every cleaning and preprocessing decision was made with an explicit bias toward preserving information and avoiding data loss, since the size of this dataset (891 rows) makes it more sensitive to row deletion than a much larger dataset would be.

9. Conclusion

This task walked through the full lifecycle of preparing a real-world dataset for analysis: acquiring the Titanic dataset from a public source, exploring its structure and quality, systematically identifying missing values, outliers, and inconsistent entries, and applying targeted cleaning and preprocessing techniques with a clear rationale for each. The final preprocessed dataset (891 rows × 24 columns) is fully numeric, free of missing values, and ready for downstream statistical analysis or machine learning, while every transformation applied has been documented here alongside the reasoning behind it.

Deliverable files

File	Description
titanic.csv	Original raw dataset (891 × 12)
titanic_cleaned.csv	After missing-value / outlier / inconsistency handling (891 × 14)
titanic_preprocessed.csv	Fully encoded & scaled, analysis-ready (891 × 24)